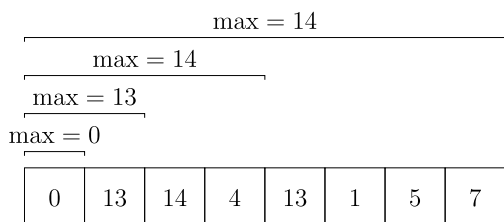


Sparse Table

ST 表

定义



ST 表（Sparse Table，稀疏表）是用于解决 **可重复贡献问题** 的数据结构。

▼ 什么是可重复贡献问题？

可重复贡献问题 是指对于运算，满足，则对应的区间询问就是一个可重复贡献问题。例如，最大值有，gcd 有，所以 RMQ 和区间 GCD 就是一个可重复贡献问题。像区间和就不具有这个性质，如果求区间和的时候采用的预处理区间重叠了，则会导致重叠部分被计算两次，这是我们所不愿意看到的。另外，还必须满足结合律才能使用 ST 表求解。

▼ 什么是 RMQ？

RMQ 是英文 Range Maximum/Minimum Query 的缩写，表示区间最大（最小）值。解决 RMQ 问题有很多方法，可以参考 [RMQ 专题](#)。

引入

[ST 表模板题](#)

题目大意：给定 n 个数，有 m 个询问，对于每个询问，你需要回答区间 $[l, r]$ 中的最大值。

考虑暴力做法。每次都对区间 $[l, r]$ 扫描一遍，求出最大值。

显然，这个算法会超时。

ST 表

ST 表基于 **倍增** 思想，可以做到 预处理，回答每个询问。但是不支持修改操作。

基于倍增思想，我们考虑如何求出区间最大值。可以发现，如果按照一般的倍增流程，每次跳 $\log$ 步的话，询问时的复杂度仍旧是 $O(n \log n)$ ，并没有比线段树更优，反而预处理一步还比线段树慢。

我们发现，也就是说，区间最大值是一个具有「可重复贡献」性质的问题。即使用来求解的预处理区间有重叠部分，只要这些区间的并是所求的区间，最终计算出的答案就是正确的。

如果手动模拟一下，可以发现我们能使用至多两个预处理过的区间来覆盖询问区间，也就是说询问时的时间复杂度可以被降至 $O(1)$ ，在处理有大量询问的题目时十分有效。

具体实现如下：

令 $st[i][j]$ 表示区间 $[i, i + 2^j - 1]$ 的最大值。

显然 。

根据定义式，第二维就相当于倍增的时候「跳了 步」，依据倍增的思路，写出状态转移方程： 。

$$\max \left(\begin{array}{|c|c|} \hline \max(\{a_i, \dots, a_{i+2^j-1}\}) & \max(\{a_{i+2^{j-1}}, \dots, a_{i+2^j-1}\}) \\ \hline \end{array} \right) \\ \parallel \\ \max(\{a_i, \dots, a_{i+2^j-1}\})$$

以上就是预处理部分。而对于查询，可以简单实现如下：

对于每个询问，我们把它分成两部分： 与 ，其中 。两部分的结果的最大值就是回答。

$$\begin{array}{c} \max = \max(\max_l, \max_r) = 14 \\ \hline \max_r = f(4, 2) = 13 \\ \hline \max_l = f(2, 2) = 14 \\ \hline \end{array}$$

0	13	14	4	13	1	5	7
---	----	----	---	----	---	---	---

根据上面对于「可重复贡献问题」的论证，由于最大值是「可重复贡献问题」，重叠并不会对区间最大值产生影响。又因为这两个区间完全覆盖了，可以保证答案的正确性。

模板代码

[ST 表模板题](#)

C 风格模板

```

1 #include <LAlgorithm>
2 #include <LTiostream>
3 using namespace std;
4 constexpr int MAXN = 2000001;
5 constexpr int logN = 21;
6 int f[MAXN][logN + 1], Logn[MAXN + 1];
7
8 void pre() { // 准备工作, 初始化
9     Logn[1] = 0;
10    Logn[2] = 1;
11    for (int i = 3; i < MAXN; i++) {
12        Logn[i] = Logn[i / 2] + 1;
13    }
14 }
15
16 int main() {
17     cin.tie(nullptr)->sync_with_stdio(false);
18     int n, m;
19     cin >> n >> m;
20     for (int i = 1; i <= n; i++) cin >> f[i][0];
21     pre();
22     for (int j = 1; j <= logN; j++)
23         for (int i = 1; i + (1 << j) - 1 <= n; i++)
24             f[i][j] = max(f[i][j - 1], f[i + (1 << (j - 1))][j - 1]); // ST表具体实现
25     for (int i = 1; i <= m; i++) {
26         int x, y;
27         cin >> x >> y;
28         int s = Logn[y - x + 1];
29         cout << max(f[x][s], f[y - (1 << s) + 1][s]) << '\n';
30     }
31     return 0;
32 }

```

C++ 风格模板

```

1 #include <LBits/stdc++.h>
2 using namespace std;
3
4 template <typename T>
5 class SparseTable {
6     using VT = vector<T>;
7     using VVT = vector<VT>;
8     using func_type = function<T(const T &, const T &)>;
9
10    VVT ST;
11
12    static T default_func(const T &t1, const T &t2) { return max(t1, t2); }
13
14    func_type op;
15
16    public:
17    SparseTable(const vector<T> &v, func_type _func = default_func) {
18        op = _func;
19        int len = v.size(), l1 = ceil(log2(len)) + 1;
20        ST.assign(len, VT(l1, 0));
21        for (int i = 0; i < len; ++i) {
22            ST[i][0] = v[i];
23        }
24        for (int j = 1; j < l1; ++j) {
25            int pj = (1 << (j - 1));
26            for (int i = 0; i + pj < len; ++i) {
27                ST[i][j] = op(ST[i][j - 1], ST[i + (1 << (j - 1))][j - 1]);
28            }
29        }
30    }
31
32    T query(int l, int r) {
33        int lt = r - l + 1;
34        int q = floor(log2(lt));
35        return op(ST[l][q], ST[r - (1 << q) + 1][q]);
36    }
37 };

```

注意点

1. 输入输出数据一般很多，建议开启输入输出优化。
2. 每次用 `std::log` 重新计算 `log` 函数值并不值得，建议进行如下的预处理：

ST 表维护其他信息

除 RMQ 以外，还有其它的「可重复贡献问题」。例如「区间按位与」、「区间按位或」、「区间 GCD」，ST 表都能高效地解决。

需要注意的是，对于「区间 GCD」，ST 表的查询复杂度并没有比线段树更优（令值域为 M ，ST 表的查询复杂度为 $O(\log M)$ ，而线段树为 $O(\log n)$ ，且值域一般是大于 n 的），但是 ST 表的预处理复杂度也没有比线段树更劣，而编程复杂度方面 ST 表比线段树简单很多。

如果分析一下，「可重复贡献问题」一般都带有某种类似 RMQ 的成分。例如「区间按位与」就是每一位取最小值，而「区间 GCD」则是每一个质因数的指数取最小值。

总结

ST 表能较好的维护「可重复贡献」的区间信息（同时也应满足结合律），时间复杂度较低，代码量相对其他算法很小。但是，ST 表能维护的信息非常有限，不能较好地扩展，并且不支持修改操作。

练习

[RMQ 模板题](#)

[「SCOI2007」降雨量](#)

[\[USACO07JAN\] 平衡的阵容 Balanced Lineup](#)

附录：ST 表求区间 GCD 的时间复杂度分析

在算法运行的时候，可能要经过 n 次迭代。每一次迭代都可能会使用 GCD 函数进行递归，令值域为 $[1, n]$ ，GCD 函数的时间复杂度最高是 $O(\log n)$ 的，所以总时间复杂度看似有 $O(n \log n)$ 。

但是，在 GCD 的过程中，每一次递归（除最后一次递归之外）都会使数列中的某个数至少减半，而数列中的数最多减半的次数为 $\log n$ ，所以，GCD 的递归部分最多只会运行 $\log n$ 次。再加上循环部分（以及最后一层递归）的 n ，最终时间复杂度则是 $O(n \log n)$ ，由于可以构造数据使得时间复杂度为 $O(n \log n)$ ，所以最终的时间复杂度即为 $O(n \log n)$ 。

而查询部分的时间复杂度很好分析，考虑最劣情况，即每次询问都询问最劣的一对数，时间复杂度为 $O(\log n)$ 。因此，ST 表维护「区间 GCD」的时间复杂度为预处理 $O(n \log n)$ ，单次查询 $O(\log n)$ 。

线段树的相应操作是预处理 $O(n \log n)$ ，查询 $O(\log n)$ 。

这并不是一个严谨的数学论证，更为严谨的附在下方：

► [更严谨的证明](#)

本页面最近更新：2024/5/24 22:18:23, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[abc1763613206](#), [Backlight](#), [CCXXI](#), [Chrogeek](#), [countercurrent-time](#), [DawnMagnet](#), [Enter-tainer](#), [firogh](#), [Fomalhauthmj](#), [Great-designer](#), [H-J-Granger](#), [Henry-ZHR](#), [hsfzLZH1](#), [iamtwz](#), [lr1d](#), [kenlig](#), [Ibdoknow](#), [leoleoasd](#), [mcendu](#), [MingqiHuang](#), [NachtgeistW](#), [orzAtalod](#), [ouuan](#), [ouuan](#), [purinliang](#), [ShadowsEpic](#), [shuzhouliu](#), [Siger Young](#), [StudyingFather](#), [SukkaW](#), [Tiphereth-A](#), [Xeonacid](#), [zymooll](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用